

Experiment # 02

Introduction to OpenCV Library w.r.t. Image Processing Basic Commands

Objective

1. To understand basics of Python OpenCV library.
2. Implementing Python array and basic commands of image processing.

Software

- Python Idle 3.7 or 3.8.5
- Spyder/VS Code/Google Colab

Theory

Open Source Computer **V**ision Library (**OpenCv**) is an open source software library written in C++ for machine learning and computer vision applications.

To install packages type following command in anaconda powershell prompt
conda install opencv
conda install numpy
conda install matplotlib

To install package in Spyder IDE, type following command
pip install opencv-python

Why OpenCV?

OpenCV is comprehensive collection of more than 2500 machine learning and computer vision algorithms that can be used from something as simple detecting faces in images to project augmented reality overlaid with scenery.

Another area in which OpenCV excels is its superior support for multiple interfaces and all the major operating systems as compared to other solutions. OpenCV supports Windows, Linux, OS X and Android and provides interfaces for C, C++, Python, Java and MATLAB.

Some of the applications that can be accomplished easily with OpenCV are: identifying objects, tracking camera movements, stitching images together, finding similar images in a database using an image, face detection and tracking moving objects in a video feed etc.

Why Numpy?

NumPy is an open-source numerical Python library. It contains a multi-dimensional array and matrix data structures. It can be utilised to perform a number of mathematical operations on arrays such as trigonometric, statistical, and algebraic routines. Therefore, the library contains a large

number of mathematical, algebraic, and transformation functions. It is an extension of Numeric and Numarray. It also contains random number generators.

```
import numpy as np
import numpy
```

There are a number of different ways to create an array. This section will provide an overview of the most common methodologies:

Create an empty array:

```
numpy.empty(2) #this will create 1D array of 2 elements
numpy.empty([2,3]) #this will create 2D array (2 rows, 3 columns
each)
```

Create an array with 0s:

```
numpy.zeros(2) #it will create an 1D array with 2 elements, both 0
#Note the parameter of the method is shape, it could be int or a
tuple
```

Create an array with 1s:

```
numpy.ones(2) # this will create 1D array with 2 elements, both 1
```

Random number generation

Use the random module of numpy for uniformly distributed numbers:

```
np.random.rand(3,2) #3 rows, 2 cols
np.random.randint()
```

Adding/Removing/Sorting Elements

We can perform a number of fast operations on a Numpy array. This makes Numpy a desirable library for the Python users.

Adding elements:

```
a = [0]
np.append(a, [1,2]) #adds 1,2 at the end
#insert can also be used if we want to insert along a given
indexThis will return [0,1,2]
```

Deleting elements:

```
np.delete(array, 1) #1 is going to be deleted from the array
e.g.
a = np.delete([0,1,2], 1) #results in [0,2]
```

Sorting

To sort an array, call the sort(array, axis, kind, orderby) function:

```
np.sort(array1)
e.g.
a = np.sort([[0,3,2],[1,2,3]])
results in
[[0 2 3]
 [1 2 3]]
```

Finding the number of dimensions of an array:

```
array.ndim
```

e.g.

```
a = np.array([[1,2],[3,4]])
```

```
print(a.ndim) shows 2
```

Add, Subtract, Multiply, Divide, Power, Mod

To perform basic arithmetic functions on two arrays a and b:

```
a = [1,2]
```

```
b= [3,4]
```

```
c = np.add(a, b)
```

```
c = np.subtract(a, b)
```

```
c = np.multiply(a, b)
```

```
c = np.divide(a, b)
```

```
c = np.power(a, 2)
```

```
c= np.mod(a, b)
```

```
c = np.remainder(a, b)
```

For two dimensional array/ images**Slice a 2D array:**

```
x = y [row_start: row_end, col_start: col_end]
```

Create a 2D array of zeros using NumPy:

```
my_array = numpy.zeros ((row, columns), dtype=numpy.uint8)
```

Create a 2D array of ones using NumPy:

```
my_array = numpy.ones ((row, columns), dtype=numpy.uint8)
```

Check the size of a 2D array:

```
size = numpy.shape(my_array)
```

Join a sequence of arrays along an existing axis:

```
F = np.concatenate ((a, b), axis=0);
```

Assemble an array from nested lists of blocks.

```
img = np.block ([[np.ones ((2, 2)), np.zeros ((2, 3))], [np.zeros ((2, 2)), np.ones ((2, 3))]])
```

Note: all the input array dimensions except for the concatenation axis must match exactly

Reading an image using OpenCv:

```
my_image = cv2.imread("test_image.jpg",0)
```

The second argument determines whether the image is read as a grayscale image or a colored image. **0** is used for reading an image as grayscale and while **1** is used for reading in color. If no argument is passed then the image is read as is.

Displaying an image using OpenCv:

```
cv2.imshow ("Title of the window", my_image).
```

Two more commands that need to be used while displaying an image are: **cv2.waitKey(x)** **cv2.destroyAllWindows ()**. The waitKey () function waits for a key being pressed for x

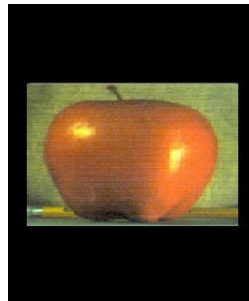
number of milliseconds. If **0** is passed to `waitKey ()` as an argument, it will wait indefinitely for a key press. **`cv2.destroyAllWindows ()`** closes all the open image windows.

Writing an image to disk:

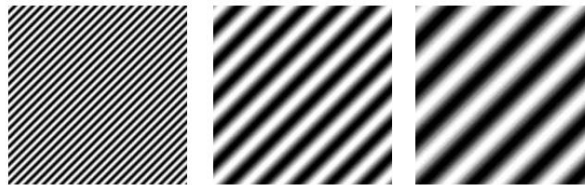
`cv2.imwrite("image_name.jpg", my_image)`

Lab Tasks

1. Now create a generic code that creates a border around any landscape image as shown below. The length of right and left borders must be 10% of the original horizontal length of the image. The length of upper and lower border must be such that the image now have same number of rows as columns. Save the image.



2. Using the following formula $f(i, j) = \sin(2\pi f(i + j))$ where i and j are indices of a pixel, draw an image with different frequencies (input from user).



3. Write Python functions that can create the images given below. Code them in such so that the size of the image itself are entered by the user.

